



Politecnico
di Bari

Multimodal AI Agent for Customer Care

Project Work Digital Business - TP Italia x Politecnico di Bari

IMPLEMENTAZIONE DATABASE SQL E CONTAINERIZZAZIONE

Obiettivo

Nel materiale presentato in precedenza, il Data Layer era stato individuato come componente incaricato di conservare conversazioni, messaggi, ticket e operatori. Nel concreto, PostgreSQL è incaricato della gestione dello schema SQL, Python mantiene un repository per l'accesso ai dati, FastAPI collega gli endpoint alla persistenza, mentre il servizio pgAdmin e Docker Compose consentono la gestione e l'avvio dell'intero sistema database.

Implementazione PostgreSQL nel backend

PostgreSQL viene introdotto come database relazionale nel backend. Il servizio è definito in *docker-compose.yml* con immagine *postgres:16-alpine*, porta interna *5432* e porta host configurabile, di default *5433*. Il volume *database_data* rende persistenti i dati anche dopo arresto e riavvio dei container. Il file *backend/db/init.sql* viene montato nella directory */docker-entrypoint-initdb.d/init.sql*, così lo schema viene creato automaticamente alla prima inizializzazione del volume.

All'avvio dell'app FastAPI, la funzione *init_database()* esegue una serie di istruzioni al fine di avviare il database e generare l'URL di connessione.

`DATABASE_URL=postgresql://tarai:ChatbotTaranto2026!@database:5432/tarai`

Schema SQL e tabelle

Le chiavi primarie sono UUID generate tramite pgcrypto, mentre le foreign key principali collegano *messages* e *tickets* alla tabella *conversations* con *ON DELETE CASCADE*, in modo da mantenere coerenza referenziale quando una conversazione viene rimossa.

Tabella	Ruolo	Campi
---------	-------	-------

operators	Operatori autorizzati all'accesso.	id UUID, email UNIQUE NOT NULL, password_hash NOT NULL, created_at.
conversations	Sessioni conversazionali del chatbot.	id UUID, ai_summary, created_at.
messages	Messaggi utente/bot associati a una conversazione.	conversation_id FK, role CHECK user/bot, type CHECK text/image/audio, satisfaction solo per bot.
tickets	Escalation verso operatore umano.	conversation_id FK, status, priority, domain, user_email, summary, ai_summary, original_message, translated_message.
kb_sources	Registro relazionale delle fonti della knowledge base.	id TEXT PK, kb_type, domain_label, title, source_url, search_text.
kb_source_domains	Aggregazione dei domini della KB per triage e analisi.	label PK, source_count, updated_at.

Persistenza delle conversazioni e dei messaggi

La persistenza è incapsulata nel modulo *persistence_repository.py*. La funzione *ensure_conversation()* garantisce l'esistenza della conversazione partendo da *conversation_id* o *session_id*.

Quando l'utente invia un messaggio al chatbot, *answer_chat()* normalizza la richiesta, assicura la conversazione e salva il messaggio utente con *save_user_message()*. Dopo l'orchestrazione LangGraph, RAG e LLM, il backend salva anche la risposta con *save_bot_message()* e restituisce al frontend *user_message_id*, *bot_message_id* e *message_id*.

La tabella *messages* conserva anche il tipo di messaggio, distinguendo text, image e audio. Nel caso di immagine o audio, il contenuto salvato può includere URL dell'upload, testo estratto da OCR, descrizione visiva o trascrizione. In questo modo la cronologia conversazionale resta utile anche per un eventuale operatore umano che deve ricostruire il contesto della richiesta.

Ticketing, feedback e KPI

La parte di ticketing consente al chatbot di passare da una risposta automatica a un flusso di customer care umano quando l'informazione non è disponibile, quando la richiesta richiede escalation o quando l'utente non è soddisfatto. L'endpoint `POST /tickets` valida l'e-mail dell'utente, assicura la conversazione, genera un triage tramite `ticket_service` e salva il ticket su PostgreSQL con *dominio*, *priorità*, *riassunto*, *messaggio originale*, *eventuale traduzione* e *ai_summary*.

Il feedback viene collegato direttamente al messaggio bot tramite `message_id`. L'endpoint `PATCH /messages/{message_id}/feedback` aggiorna il campo `satisfaction` solo per messaggi con `role = bot`. Questa scelta evita dati ambigui, perché la valutazione deve riferirsi alla risposta generata dal sistema e non al messaggio inviato dall'utente.

L'endpoint `GET /kpis` calcola indicatori aggregati direttamente da PostgreSQL, in termini di *totale conversazioni*, *totale messaggi*, *messaggi user/bot*, *ticket totali*, *ticket open/in_progress/closed*, *feedback positivi*, *feedback negativi*, *messaggi valutati* e *satisfaction_rate*.

pgAdmin

pgAdmin è stato inserito nel `docker-compose.yml` come servizio dedicato con immagine `dpape/pgadmin4:latest` e porta host `5050`. Dipende dal servizio database e si avvia solo quando PostgreSQL risulta healthy.

Containerizzazione con Docker Compose

Docker Compose funge come punto di ingresso unico del sistema. Il file `docker-compose.yml` definisce i servizi *frontend*, *backend*, *vector-db*, *database*, *pgAdmin*, *llm*, *llm-init* e *cloudflared*. Tutti i container condividono la rete *tarai-network* e i servizi comunicano tramite nomi DNS interni. Questa struttura rende il progetto più semplice da avviare su macchine diverse, riducendo la dipendenza da configurazioni manuali locali.

Servizio	Tecnologia	Porta	Responsabilità
frontend	React/Vite	5173	Interfaccia chat, PWA, upload audio/immagini, lingua e tema.
backend	FastAPI	8000	API applicative, RAG, LLM, multimodalità, persistenza.

vector-db	ChromaDB	8001	Vector database per retrieval semantico della KB.
database	PostgreSQL 16	5433 host / 5432 container	Persistenza relazionale di conversazioni, messaggi, ticket e KPI.
pgadmin	pgAdmin4	5050	Ispezione e gestione visuale del database.
llm	Ollama	11434	Esecuzione locale dei modelli LLM.
cloudflared	Cloudflare Tunnel	HTTPS temporaneo	Test da smartphone/PWA/microfono su tunnel pubblico temporaneo.

Dockerfile backend e frontend

Il Dockerfile del backend parte da *python:3.11-slim*, imposta */app* come working directory, abilita variabili per *logging* e cache *HuggingFace*, installa dipendenze di sistema necessarie alla pipeline multimodale e poi installa i *requirements* Python. La presenza di *ffmpeg* supporta i messaggi audio, mentre *tesseract-ocr* e *tesseract-ocr-ita* supportano l'OCR su immagini e documenti. Il container espone la porta *8000* e avvia *Uvicorn*.

Il Dockerfile del frontend parte da *node:22-alpine*, installa le dipendenze con *npm* e avvia il *dev server Vite* sulla porta *5173*.